

Terminal Simulator - Project Report

Team Members:

- Abdelrahman Diab (202444200)
 - Qais Abu Adas (202465300)
-

How it works

This is a simple terminal that simulates a file system. You can create folders, make files, navigate around, and search for text just like a real terminal.

How it is built

The File System Structure

We used a tree structure where everything is a Node. A node can be either:

- A File (has content and size)
- A Directory (can contain other files and directories)

This is the Composite Pattern or let's say Polymorphism, where directories and files share the same base so we can treat them in a unified way.

Why We Picked Each Data Structure

HashMap for Directory Children

Where: Inside each Directory

Why: Built-in easy, I do not need to handle probing or so. Furthermore, When you type

commands like `cd` Documents or `ls`, we need to find items fast. A HashMap gives $O(1)$ time compared to other options.

Stack for pwd Command

Where: Building the current path

Why: When you're in a deep path like `/home/test/Documents`, we need to walk back to root and build the path. A stack is perfect since items are pushed as we go up and popped to print in correct order.

Tree for the Whole System

Where: The entire file system

Why: File systems are hierarchical. A tree models this naturally, with each directory knowing its parent and children.

KMP Algorithm for grep

Where: Searching text in files

Why: KMP (Knuth-Morris-Pratt) avoids redundant comparisons by remembering previous matches, making it far better than the brute-force algorithm.

Main Commands Explained

Command	What Happens Inside
mkdir	Creates a new Directory node and adds it to the parent' s HashMap
touch	Creates a new File node with the given size
cd	Walks the tree using HashMap lookups to update the current directory
ls	Iterates through the current directory' s children
pwd	Uses a stack to build the full path from current directory to root
rm	Removes a node from the parent' s HashMap
tree	Recursively prints directory contents with indentation
du	Recursively sums sizes of all files under a directory
grep	Runs the KMP algorithm on file content

Class Breakdown

- Node – Base class (abstract)
 - File – Stores content and size, supports KMP search
 - Directory – Stores children using a HashMap
 - FileSystem – Implements all commands such as mkdir, cd, rm, etc.
 - TermFS – Parses user input and calls FileSystem operations
 - LabStack – Stack implementation using ArrayList
 - KMP – Pattern matching algorithm
 - FileSystemException – Custom error handling
-

Quick Flow

User types command → TermFS parses it → FileSystem executes → Tree structure updates
Out of this md, make a well designed pdf report